

Great 🎯 Let's move into **Module 22: Functions**.

SQL CLASS NOTE – MODULE 22: FUNCTIONS

1. Introduction

In SQL, a **Function** is a **stored program** that performs a calculation or returns a single value.

Unlike **procedures**, functions:

-  Always return a **value** (scalar or table).
-  Can be used in **queries (SELECT, WHERE, ORDER BY, etc.)**.
-  Cannot perform transactions (COMMIT/ROLLBACK).

2. Types of Functions

1. **Built-in Functions** – provided by SQL itself.

- String functions: UPPER() , LOWER() , CONCAT()
- Date functions: NOW() , YEAR() , DATEDIFF()
- Math functions: ROUND() , ABS() , POWER()
- Aggregate functions: COUNT() , AVG() , SUM()

2. **User-Defined Functions (UDFs)** – created by us to meet business needs.

3. Syntax of a User-Defined Function

```
CREATE FUNCTION function_name (parameters)
RETURNS datatype
DETERMINISTIC
BEGIN
    -- SQL logic
    RETURN value;
END;
```

- **RETURNS datatype** → specifies the return type.
- **DETERMINISTIC** → means the function always returns the same output for the same input.

4. Practical Examples with SchoolDB

Example 1: Function to get full student name

```
CREATE FUNCTION GetFullName(sid INT)
RETURNS VARCHAR(100)
DETERMINISTIC
BEGIN
    DECLARE full_name VARCHAR(100);
    SELECT CONCAT(FirstName, ' ', LastName) INTO full_name
    FROM Students WHERE StudentID = sid;
    RETURN full_name;
END;

-- Usage
SELECT GetFullName(101);
```

Example 2: Function to calculate student age in years

```
CREATE FUNCTION GetStudentAge(sid INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE age INT;
    SELECT Age INTO age FROM Students WHERE StudentID = sid;
    RETURN age;
END;

-- Usage
SELECT FirstName, LastName, GetStudentAge(StudentID) AS Age
FROM Students;
```

Example 3: Function to count students in a grade

```
CREATE FUNCTION CountByGrade(grade VARCHAR(20))
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE total INT;
    SELECT COUNT(*) INTO total
    FROM Students
    WHERE GradeLevel = grade;
    RETURN total;
END;

-- Usage
SELECT CountByGrade('Grade 10') AS Total_Grade10;
```

Example 4: Function to calculate GPA of a student

👉 Assuming Enrollments table has Grade column (A=4, B=3, C=2, D=1, F=0).

```
CREATE FUNCTION GetGPA(sid INT)
RETURNS DECIMAL(3,2)
DETERMINISTIC
BEGIN
    DECLARE gpa DECIMAL(3,2);
    SELECT AVG(
        CASE Grade
            WHEN 'A' THEN 4
            WHEN 'B' THEN 3
            WHEN 'C' THEN 2
            WHEN 'D' THEN 1
            ELSE 0
        END
    ) INTO gpa
    FROM Enrollments
    WHERE StudentID = sid;
    RETURN gpa;
END;

-- Usage
SELECT StudentID, GetGPA(StudentID) AS GPA FROM Students;
```

Example 5: Function to check if a student is enrolled in any course

```
CREATE FUNCTION IsEnrolled(sid INT)
RETURNS VARCHAR(10)
DETERMINISTIC
BEGIN
    DECLARE result VARCHAR(10);
    IF EXISTS (SELECT 1 FROM Enrollments WHERE StudentID = sid) THEN
        SET result = 'YES';
    ELSE
        SET result = 'NO';
    END IF;
    RETURN result;
END;

-- Usage
SELECT StudentID, GetFullName(StudentID), IsEnrolled(StudentID) AS Enrolled
FROM Students;
```

5. Why Use Functions?

-  Reuse SQL logic instead of rewriting it.
-  Improve code readability.
-  Encapsulate complex calculations (like GPA).
-  Make queries shorter and easier to maintain.

6. Real-Life Use Cases

- **Schools:** Calculate GPA, student counts, attendance percentage.
- **Banks:** Compute interest, loan eligibility, account balances.
- **E-commerce:** Calculate discount price, loyalty points.
- **Healthcare:** Calculate patient age from birthdate, BMI.

7. Classwork (Practice)

1. Create a function that returns a student's full name.
2. Create a function that returns a student's age.
3. Create a function that counts students in a grade.
4. Create a function that calculates a student's GPA.
5. Create a function that checks if a student is enrolled.

8. Challenge Question

 Create a function called `ScholarshipEligibility(sid)` that returns **“Eligible”** if the student's GPA ≥ 3.5 , otherwise returns **“Not Eligible”**.

 That's the **expanded Module 22: Functions**.

Got it  Let's handle the **solutions for Module 22: Functions** first, then I'll tell you how many modules are left.



Solutions to Module 22 Classwork + Challenge

1. Function to return a student's full name

```
CREATE FUNCTION fn_GetFullName(sid INT)
RETURNS VARCHAR(100)
DETERMINISTIC
BEGIN
    DECLARE full_name VARCHAR(100);
    SELECT CONCAT(FirstName, ' ', LastName) INTO full_name
    FROM Students WHERE StudentID = sid;
    RETURN full_name;
END;

-- Usage
SELECT fn_GetFullName(101);
```

2. Function to return a student's age

```
CREATE FUNCTION fn_GetAge(sid INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE age INT;
    SELECT Age INTO age FROM Students WHERE StudentID = sid;
    RETURN age;
END;

-- Usage
SELECT fn_GetAge(105);
```

3. Function to count students in a grade

```
CREATE FUNCTION fn_CountByGrade(grade VARCHAR(20))
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE total INT;
    SELECT COUNT(*) INTO total
    FROM Students
    WHERE GradeLevel = grade;
    RETURN total;
END;

-- Usage
SELECT fn_CountByGrade('Grade 11') AS Total_Grade11;
```

4. Function to calculate a student's GPA

```
CREATE FUNCTION fn_GetGPA(sid INT)
RETURNS DECIMAL(3,2)
DETERMINISTIC
BEGIN
    DECLARE gpa DECIMAL(3,2);
    SELECT AVG(
        CASE Grade
            WHEN 'A' THEN 4
            WHEN 'B' THEN 3
            WHEN 'C' THEN 2
            WHEN 'D' THEN 1
            ELSE 0
        END
    ) INTO gpa
    FROM Enrollments
    WHERE StudentID = sid;
    RETURN gpa;
END;

-- Usage
SELECT StudentID, fn_GetGPA(StudentID) AS GPA FROM Students;
```

5. Function to check if a student is enrolled

```
CREATE FUNCTION fn_IsEnrolled(sid INT)
RETURNS VARCHAR(10)
DETERMINISTIC
BEGIN
    DECLARE result VARCHAR(10);
    IF EXISTS (SELECT 1 FROM Enrollments WHERE StudentID = sid) THEN
        SET result = 'YES';
    ELSE
        SET result = 'NO';
    END IF;
    RETURN result;
END;

-- Usage
SELECT StudentID, fn_GetFullName(StudentID), fn_IsEnrolled(StudentID) AS Enrolled
FROM Students;
```



Challenge Question

👉 Scholarship eligibility based on $\text{GPA} \geq 3.5$


```
CREATE FUNCTION fn_ScholarshipEligibility(sid INT)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    DECLARE eligibility VARCHAR(20);
    DECLARE gpa DECIMAL(3,2);

    SET gpa = fn_GetGPA(sid);

    IF gpa >= 3.5 THEN
        SET eligibility = 'Eligible';
    ELSE
        SET eligibility = 'Not Eligible';
    END IF;

    RETURN eligibility;
END;

-- Usage
SELECT StudentID, fn_GetFullName(StudentID), fn_ScholarshipEligibility(StudentID) AS Scholarsh:
FROM Students;
```

✅ That covers **all the classwork and challenge for Functions**.

Remaining Modules

From the screenshot structure you uploaded earlier, we have **25 modules in total**.

- We just finished **Module 22: Functions**.
- That means we have **3 modules left**:
 - **Module 23: Transactions**
 - **Module 24: Views**
 - **Module 25: Indexes**